

ARCADIA STATION

AI 서버 연동 백엔드 Q&A;

NPC 심문, RAG 사건기록 검색, CaseBlueprint, 인증, 배포, 콜드 스타트, 성능과 폴링 계약

문서 상태	기준	작성일
백엔드 전달용	현재 구현 + 스테이징 전 권장 보완	2026-07-28

먼저 공유할 핵심 결론

현재 API 계약은 로컬 및 통합 테스트에서 동작한다. 그러나 NPC/RAG 인증은 아직 사건 생성 API와 통일되지 않았고, 세션과 CaseBlueprint가 메모리에만 저장된다. 따라서 이 문서는 '지금 호출 가능한 계약'과 '스테이징 전 반드시 보완할 운영 계약'을 구분한다.

작성 근거: AI 서버 저장소의 현재 Java 코드, JSON Schema, 전체 READY 응답 샘플, 22개 자동화 테스트, 실제 Gemini 호출 측정 결과, Render 공식 문서.

한 페이지 요약

질문	현재 답변	판정
1. NPC/RAG 경로	현재 구현 경로가 존재한다. 단, 두 경로는 아직 X-Internal-AI-Key를 검사하지 않는다.	사용 가능 / 보안 보완 필요
2. 컨텍스트 전달	정상 동작 중인 같은 AI 서버 인스턴스라면 sessionId와 질문 입력만 보낸다.	sessionId 중심
3. JSON Schema	Draft 2020-12 원본 파일을 별도 첨부한다. 추가 필드는 거부하도록 정의되어 있다.	첨부 완료
4. 내부 키	현재 사건 생성/조회에만 적용된다. 자동 발급이나 로테이션 기능은 없다.	수동 공유 비밀
5. Base URL	현재 확정된 주소는 로컬 개발 주소뿐이다. 스테이징/운영 URL은 아직 없다.	배포 미정
6. 콜드 스타트	현재 로컬에는 없다. Render Free 사용 시 idle spin-down이 있고, 메모리 세션 손실이 더 큰 위험이다.	DB 전환 필요
7. 생성 시간	실제 Gemini 구조화 응답 55.39초. 기본 최대 3회이므로 외부 호출 대기는 최악 약 180초다.	1.5초 폴링 허용

실제 검증 범위

자동화 테스트 22개가 모두 통과했다. 실제 Gemini 임베딩 호출은 성공했고, 큰 사건 구조화 응답도 55.39초에 수신 및 JSON Schema 파싱까지 성공했다. 다만 그 사건은 서버 게임 규칙 8건을 위반해 최종적으로 검증된 FALLBACK 사건으로 교체됐다. 즉 통신과 구조화 응답은 실측 성공, AI 생성 사건의 최종 규칙 통과는 아직 미확인이다.

스테이징 전 필수 결정 4건

- NPC/RAG 내부 인증 통일: 현재 공개 컨트롤러 경로를 그대로 보호할지, /internal/v1 하위 별칭을 만들지 결정한다.
- 세션 영속화: InMemoryGameSessionRepository를 DB/Redis로 교체하거나 CaseBlueprint 재주입 API를 만든다.
- Render 서비스 유형: 백엔드와 AI를 같은 workspace + region에 두고 AI는 Private Service로 운영하는 안을 권장한다.
- 시간 예산: 사건 준비 총 210초, 폴링 2초 권장, NPC/RAG는 별도 짧은 제한을 실제 코드에 적용한다.

01

NPC 심문과 RAG 사건기록 검색의 경로와 요청/응답 필드

현재 코드에서 호출 가능한 정확한 경로

NPC와 RAG는 SessionController의 /api/v1/sessions 하위에 구현되어 있다. 사건 생성/상태 조회의 /internal/v1/cases와 네임스페이스가 다르다.

NPC 심문

POST /api/v1/sessions/{sessionId}/interrogations/{characterId}/turns
Content-Type: application/json

```
{
  "question": "안전 점검 기록을 설명해 주세요.",
  "presentedClueIds": ["CLUE-SETUP-PANEL", "CLUE-TRIGGER-LOG"]
}
```

```
{
  "dialogue": "그 기록이 있다면 일부는 인정하죠. 사건 전 소피아 인증으로 의료 안전 점검 예약이 등록됐습니다.",
  "emotion": "DEFENSIVE",
  "revealedFactIds": ["FACT-SETUP"],
  "recommendedQuestions": [
    {"topicId": "TOPIC-1", "label": "안전 점검 예약의 목적"},
    {"topicId": "TOPIC-2", "label": "02:05 진단 실행 시각"}
  ]
}
```

필드	타입	규칙
question	string, 필수/공백 불가	플레이어 질문
presentedClueIds	string[], 필수	이 세션에서 이미 발견된 단서만 허용
dialogue	string	NPC 대사
emotion	enum	CALM, DEFENSIVE, ANXIOUS, ANGRY, EVASIVE
revealedFactIds	string[]	서버가 계산한 revealableFactIds의 부분집합
recommendedQuestions	object[2]	각 항목은 topicId, label

RAG 사건기록 검색

POST /api/v1/sessions/{sessionId}/assistant/queries
Content-Type: application/json

```
{
  "question": "02:05 소피아 안전 진단 기록을 보여줘"
}
```

```
{
  "answer": "02:05 의료 안전 진단 실행 기록: 02:05 생명 유지 시스템에서 의료 안전 진단이 실행됐다...",
  "citedRecordIds": ["RECORD-TRIGGER"],
  "suggestedQueries": [
    "02:05 관련 기록을 더 보여줘",
    "소피아 관련 기록을 더 보여줘"
  ],
  "newlyDiscoveredClues": [
    {
      "clueId": "CLUE-TRIGGER-LOG",
      "title": "02:05 안전 진단 감사 로그",
      "clueType": "DIGITAL",
      "solutionRoles": ["TRIGGER"],
      "playerText": "02:05에 소피아의 인증으로 의료 안전 진단 작업이 실행됐다."
    }
  ]
}
```

RAG 응답 필드와 공통 오류

필드	타입	설명
answer	string	검색된 확정 기록만 사용한 요약
citedRecordIds	string[]	이번 응답의 근거가 된 EvidenceRecord ID
suggestedQueries	string[]	후속 검색어, AI Schema는 2개를 요구
newlyDiscoveredClues	PublicClueView[]	검색 결과로 새로 획득한 공개 단서
PublicClueView.clueId	string	실제 AI 단서 ID
PublicClueView.clueType	enum	PHYSICAL, DIGITAL, MOTIVE, OPPORTUNITY
PublicClueView.solutionRoles	enum[]	백엔드 내부 추리 역할. 브라우저 노출 정책은 별도 적용

HTTP	code	발생 조건
400	INVALID_REQUEST	빈 질문, 존재하지 않는 characterId, 미발견 단서 제시
404	SESSION_NOT_FOUND	AI 서버 메모리에 sessionId가 없음
409	SESSION_NOT_READY	CaseBlueprint 동결 전 NPC/RAG 호출
409	INVALID_SESSION_STATE	세션 상태 전이 위반

보안 주의

현재 NPC/RAG 엔드포인트에는 X-Internal-AI-Key 검사가 없다. 개발 호출은 가능하지만, public URL로 스테이징 배포하기 전에는 동일한 내부 인증 필터를 적용하거나 /internal/v1/cases/{sessionId}/... 경로를 추가해야 한다.

02

CaseBlueprint 보관과 요청별 컨텍스트 전달 책임

직접 답변

현재 MVP에서는 사건 생성 완료 후 AI 서버가 FrozenCaseBlueprint, 발견 단서 목록, RAG 인덱스를 세션별로 보관한다. 따라서 백엔드는 허용된 사실 목록을 매 요청마다 보내지 않는다. NPC에는 sessionId + characterId + question + presentedClueIds, RAG에는 sessionId + question만 보낸다.

AI 서버가 내부에서 조립하는 컨텍스트

영역	AI 서버 내부 데이터	백엔드가 보내는 값
NPC	npcKnowledge, known/concealed facts, revealPolicies, discovered evidence	question, presentedClueIds
RAG	evidenceRecords, 임베딩 인덱스, 현재 evidenceInventory	question
검증	세계관/규칙 템플릿, JSON Schema, 동결 SHA-256	sessionId

게임 백엔드

-> POST /internal/v1/cases {sessionId, seed}

AI 서버

-> 사건 생성 + 서버 검증 + FrozenCaseBlueprint 저장 + RAG 인덱스 생성

게임 백엔드

-> NPC: sessionId / characterId / question / presentedClueIds

-> RAG: sessionId / question

AI 서버

-> sessionId로 사건과 진행 상태를 조회해 안전 컨텍스트를 자체 조립

현재 운영상 가장 큰 제약

보관소는 ConcurrentHashMap 기반 InMemoryGameSessionRepository다. AI 서버가 재시작, 재배포, 콜드 스타트되면 CaseBlueprint와 evidenceInventory가 모두 사라진다. 백엔드가 CaseBlueprint를 보관해도 현재 AI API에는 이를 다시 주입해 세션을 복원하는 엔드포인트가 없다.

스테이징 전 권장 구조

- AI 서버가 PostgreSQL/Redis에 FrozenCaseBlueprint, 세션 상태, evidenceInventory를 영속화한다.
- 또는 백엔드 저장본을 AI 서버에 재주입하는 idempotent rehydrate API를 추가한다.
- allowedFacts 같은 내부 권한 컨텍스트는 백엔드 입력으로 받지 않는다. 클라이언트가 이를 위조하면 NPC 비밀 누출 위험이 생긴다.
- 백엔드는 원본 CaseBlueprint를 백업/감사 목적으로 저장하되, AI 호출 시에는 최소 입력만 전송한다.

03

case-blueprint.schema.json 원본 공유와 역직렬화 기준

전달 파일

이 PDF와 같은 폴더에 case-blueprint.schema.json과 internal-case-ready.response.json을 함께 준비했다. GitHub 원본: case-blueprint.schema.json / 전체 READY 응답

최상위 CaseBlueprint 필드

필드	타입	설명
blueprintId	string	사건 ID
seed	string	재현용 seed
worldTemplate	TemplateRef	{id, version}
ruleTemplate	TemplateRef	{id, version}
culpritId	string	범인 characterId
title	string	사건 제목
briefing	string	플레이어 공개 브리핑
truthSummary	string	백엔드 전용 전체 진실
method	Method	준비/축발 액션과 피해 조건
timeline	TimelineEvent[]	시간순 사건
facts	Fact[]	확정 사실/주장
alibis	Alibi[]	최초 주장과 실제 동선
clues	Clue[]	단서, 획득 조건, 공개문
evidenceRecords	EvidenceRecord[]	RAG 검색 원문
npcKnowledge	NpcKnowledge[]	NPC 지식/은폐/해금
redHerrings	RedHerring[]	미끼 단서
solution	Solution	정답 증거와 비범인 배제

중첩 모델

모델	필드
TemplateRef	id, version
Method	fictionalSummary, setupAction, triggerAction, victimCondition
CaseAction	actorId, locationId, systemId, operation, requiredCapabilityIds[]
TimelineEvent	eventId, time, actorIds[], locationId, actionType, summary, factIds[]
Fact	factId, kind, statement, truthValue, subjectCharacterIds[]
Alibi	characterId, initialClaim, actualWhereabouts, supportingFactIds[], contradictingFactIds[]
Clue	clueId, title, clueType, isCore, solutionRoles[], source, acquisition, revealsFactIds[], playerText, suspectEffects[]
Acquisition	type, locationId?, characterId?, requiredClueIds[], queryTopics[]
EvidenceRecord	recordId, recordType, timestamp, title, body, metadata, revealsClueIds[], searchTerms[], visibility
NpcKnowledge	characterId, knownFactIds[], initialClaimFactIds[], concealedFactIds[], revealPolicies[], allowedLieFactIds[], recommendedQuestionTopics[]
RevealPolicy	factId, requiredPresentedClueIds[], responseMode
RedHerring	redHerringId, suspectId, presentation, resolutionFactIds[], mustBeResolvable
Solution	culpritId, requiredEvidenceByRole, acceptedAlternativesByRole, nonCulpritExclusions[]
NonCulpritExclusion	characterId, excludedByClueIds[], reason

열거형

enum	값
EvidenceRole	SETUP, TRIGGER, OPPORTUNITY, MOTIVE, VICTIM_CONDITION
ClueType	PHYSICAL, DIGITAL, MOTIVE, OPPORTUNITY
AcquisitionType	EXPLORE, INTERROGATE, RAG_QUERY, CONNECT, AUTO
SuspectEffectType	SUPPORTS, EXCLUDES, NEUTRAL
TimelineActionType	MOVEMENT, SYSTEM_ACTION, CONVERSATION, DISCOVERY, BACKGROUND
FactKind	ACTION, MOTIVE, ALIBI, EXCLUSION, CONDITION, CLAIM
RecordVisibility	SEARCHABLE, HIDDEN
ResponseMode	DENIAL, EVASION, PARTIAL_ADMISSION, FULL_ADMISSION

역직렬화 규칙

Schema는 JSON Schema Draft 2020-12이며 주요 object에 additionalProperties=false가 적용된다. nullable 필드는 acquisition.locationId와 acquisition.characterId이며 string 또는 null이다. Java 기준 EvidenceRecord.metadata는 Map, Solution의 증거 역할 맵은 Map>로 매핑된다.

첨부 무결성 - schema SHA-256: 0d4544262bd279d2c910453e68679cf1c13c77e688c51a96cf5df3715045deca
READY sample SHA-256: 911f7af46e82ec311c2837bb14450fe6e31eba81c9b54ae03eb27227e681e4eb

04

X-Internal-AI-Key 적용 범위, 발급, 보관과 로테이션

엔드포인트	현재 키 검사	현재 상태
POST /internal/v1/cases	예	AI_INTERNAL_API_KEY와 일치해야 함
GET /internal/v1/cases/{sessionId}	예	AI_INTERNAL_API_KEY와 일치해야 함
POST .../interrogations/.../turns	아니오	현재 공개 컨트롤러
POST .../assistant/queries	아니오	현재 공개 컨트롤러

직접 답변

현재는 세 엔드포인트 공통 키가 아니다. 사건 생성/조회에만 적용된다. 키는 외부 기관이 발급하는 토큰이 아니라 두 서버 운영자가 생성해 AI 서버의 AI_INTERNAL_API_KEY와 게임 백엔드의 secret에 동일하게 넣는 pre-shared secret이다.

현재 동작

- AI_INTERNAL_API_KEY가 비어 있으면 사건 내부 API도 인증 검사를 사실상 생략한다.
- 잘못된 키는 403 Forbidden이다.
- 키 자동 발급 API, 만료일, 로테이션 스케줄, 이중 키 허용 로직은 아직 없다.
- 브라우저/Vite 환경변수에는 절대 넣지 않고 서버 secret으로만 보관한다.

스테이징 권장 계약

항목	권장안
적용 범위	사건 생성/조회, NPC, RAG 모두 같은 내부 인증 필터 적용
환경 분리	staging/prod에 서로 다른 32바이트 이상 랜덤 키
보관	Render Environment Group 또는 Secret Manager
전송	사설망 + X-Internal-AI-Key, 로그/응답에 키 출력 금지
로테이션	AI가 current/previous 두 키를 잠시 허용 -> 백엔드 교체 -> previous 제거
주기	정책 합의 필요. 기본 90일 또는 사고/퇴사/노출 즉시 교체 권장

05

실제 base URL, staging/production, Render 배포와 Private Network

현재 확정 사실

저장소에는 render.yaml, Dockerfile, Procfile, staging/prod application profile이 없다. 따라서 실제 staging/prod base URL은 아직 존재하지 않는다. 문서상 로컬 권장 주소는 게임 백엔드 `http://127.0.0.1:8080`, AI 서버 `http://127.0.0.1:8081`이다.

권장 Render 토폴로지

```
Render workspace (same region)
production environment
  game-backend [Web Service]
    |
    | private hostname + X-Internal-AI-Key
    v
  game-ai      [Private Service]

staging environment
  별도 backend / AI 서비스 + 별도 키 + 별도 데이터 저장소
```

- Render 공식 문서상 같은 workspace와 같은 region의 서비스는 shared private network로 통신할 수 있다.
- AI 서버를 Private Service로 만들면 public onrender.com URL 없이 백엔드에서만 접근할 수 있다.
- 내부 주소는 Dashboard의 Connect > Internal에 표시되는 hostname과 port를 사용한다.
- Free Web Service는 private network 요청을 보낼 수 있지만 받을 수 없다. 따라서 AI 수신 서비스로 Free Web을 쓰는 안은 사설망 요구와 맞지 않는다.
- 현재 앱은 Render의 PORT를 server.port로 연결하는 설정이 없다. 배포 전 application.yml에 `server.port=${PORT:8080}`을 추가하거나 start command로 `-Dserver.port=$PORT`를 전달해야 한다.

환경	AI_BASE_URL 값	비고
local	<code>http://127.0.0.1:8081</code>	현재 문서상 권장
staging	Render Internal URL - 미정	서비스 생성 후 확정
production	Render Internal URL - 미정	staging과 분리

답변 문구

현재 AI 서버는 아직 Render에 배포되지 않아 staging/prod URL은 미정입니다. 배포 시 백엔드와 같은 Render workspace 및 region에 AI Private Service로 구성하면 Private Network 통신이 가능합니다. 실제 internal hostname은 서비스 생성 후 공유하겠습니다.

06

콜드 스타트, 슬립, 재시작과 백엔드 타임아웃

환경	콜드 스타트/슬립	현재 판단
현재 로컬	없음	프로세스가 켜져 있는 동안만 동작
Render Free Web	15분 inbound idle 후 spin-down, 재기동 약 1분	테스트용 가능, 사설 수신 불가
Render Private Service	Free Web의 idle spin-down 대상이 아님	권장 운영 형태
모든 Render 서비스	재배포/장애/플랫폼 재시작 가능	메모리 세션 손실 대비 필수

콜드 스타트보다 더 중요한 현재 위험

세션, 동결 사건, 발견 단서, RAG 인덱스가 모두 메모리다. 어떤 이유로든 AI 프로세스가 재시작되면 기존 sessionId는 404가 된다. 영속화 전에는 운영 배포가 안전하지 않다.

권장 백엔드 시간 예산

호출	권장 timeout	재시도
POST /internal/v1/cases	연결 포함 10초	202 미수신 시 먹등 sessionId로 제한적 재시도
GET 상태 조회	호출당 5초	다음 폴링에서 계속
NPC 심문	목표 10~12초	먹등 키 도입 전 자동 재전송 주의
RAG 검색	목표 10~12초	실패 시 서버 결정적 요약/키워드 검색
전체 사건 준비	210초	초과 시 오류가 아니라 fallback READY 여부 먼저 확인

현재 코드와 목표값의 차이

application.yml에는 npc.timeout=8s가 있으나 실제 Gemini RestClient는 caseGeneration.timeout(60s)을 공통 read timeout으로 사용한다. 즉 NPC/RAG도 최악 60초까지 기다릴 수 있다. 스테이징 전 목적별 HTTP client timeout을 분리해야 위 목표값을 보장할 수 있다.

Render 근거: Free instances, Private Network, Private Services.

07

사건 생성 시간, 최대 시간과 상태 조회 폴링 주기

실측과 설정

항목	값	해석
실제 Gemini 구조화 응답	55.39초	JSON Schema 파싱 성공 1회
기존 20초 설정	6회 모두 실패	호출당 약 20.2초로 timeout과 일치
현재 per-attempt timeout	60초	AI_CASE_GENERATION_TIMEOUT
현재 maxRetries	2	최초 1회 + 추가 2회 = 총 3회
외부 호출 최악 대기	약 180초	60초 x 3회, 로컬 검증/인덱싱 제외
최종 결과	READY / FALLBACK 가능	AI 실패나 규칙 불일치여도 게임 시작 가능

보통 몇 초인가?

현재 실측 표본은 큰 Gemini 사건 1건뿐이므로 통계적 '보통' 값은 아직 말할 수 없다. 확인된 값은 55.39초다. 최소 30건 이상의 staging 측정으로 p50/p95를 만든 뒤 SLA를 확정해야 한다.

폴링 답변

- 현재 GET /internal/v1/cases/{sessionId}에는 rate limit이 구현되어 있지 않아 1.5초 간격도 기술적으로 허용된다.
- 단일 세션 기준 권장 기본값은 2초 + 0~300ms jitter다. 30초 이후에는 3초, 90초 이후에는 5초로 backoff하면 불필요한 트래픽을 줄일 수 있다.
- Retry-After 또는 pollAfterMs 헤더/필드는 아직 없다. 백엔드에서 자체 정책을 적용한다.
- CREATING 또는 VALIDATING 중 generation은 null이다. READY에서만 전체 generation이 들어온다.
- generationSource=FALLBACK도 정상 READY다. 백엔드는 실패로 취급하지 말고 같은 DTO로 저장한다.

권장 폴링 예시

0~30초 : 2.0초 + jitter
 30~90초 : 3.0초 + jitter
 90~210초 : 5.0초 + jitter
 210초 이후 : 마지막 상태 확인 후 timeout 처리 및 운영 로그 기록

1.5초 제안에 대한 최종 답

현재 서버 제한은 없으므로 1.5초도 가능하다. 다만 2초 + jitter를 권장한다. 1.5초를 유지해도 서버에는 메모리 조회 1회뿐이라 기능상 문제는 없지만, 다중 세션 동시 시작을 고려하면 backoff가 더 안전하다.

백엔드에 전달할 최종 합의문

현재 바로 구현해도 되는 부분

사건 생성/상태 조회 DTO, CaseBlueprint 역직렬화 모델, NPC/RAG 요청/응답 DTO, 상태 폴링 로직은 본 문서와 첨부 JSON 기준으로 구현할 수 있다.

권장 전달 메시지

현재 구현 기준으로 NPC 심문은

POST /api/v1/sessions/{sessionId}/interrogations/{characterId}/turns,

RAG 검색은

POST /api/v1/sessions/{sessionId}/assistant/queries 입니다.

AI 서버가 sessionId별 CaseBlueprint와 조사 상태를 보관하므로, NPC에는 question/presentedClueIds, RAG에는 question만 보내면 됩니다. allowedFacts 등 내부 컨텍스트는 AI 서버가 자체 조립합니다.

정확한 모델 생성 기준은 함께 전달드린 case-blueprint.schema.json, 전체 응답 기준은 internal-case-ready.response.json을 사용해 주세요.

단, 현재 NPC/RAG의 내부 키 적용과 세션 영속화, 실제 Render URL은 스테이징 전 확정/보완 항목입니다. 확정 전에는 public production 계약으로 간주하지 말아 주세요.

전달 패키지

파일	용도	SHA-256
arcadia-ai-backend-integration-qa-ko.pdf	7개 질문 상세 답변	이 문서
case-blueprint.schema.json	DTO/역직렬화 원본	0d4544262bd279d2c910453e68679cf1c13c77e688c51a96cf5df3715045deca
internal-case-ready.response.json	필드 전체가 든 READY 실제 응답	911f7af46e82ec311c2837bb14450fe6e31eba81c9b54ae03eb27227e681e4eb

스테이징 전 AI 개발 작업 체크리스트

우선순위	작업	완료 기준
P0	NPC/RAG 내부 인증 통일	세 엔드포인트 모두 동일 인증 테스트 통과
P0	세션/사건/발견 단서 영속화	재시작 후 같은 sessionId로 NPC/RAG 성공
P0	Render Private Service 배포	internal URL + health + secret 분리
P1	목적별 timeout 분리	NPC 8초, RAG 목표값, 사건 60초 별도 적용
P1	폴링 계약 필드화	pollAfterMs 또는 Retry-After 제공
P1	staging 30건 성능 측정	p50, p95, fallback rate 보고
P2	내부 키 이중 로테이션	current/previous 무중단 교체

외부 참고 자료

- render.com/docs/private-network | render.com/docs/private-services

- render.com/docs/free | render.com/docs/health-checks